

Detailed Reference Manual for the Educational Computer Instruction Set for Laboratory Work

Site: Educational Portal "Electronic University VSU"

Course: Computer Architecture (Tolstobrov A.P., Vetokhin V.V.)

Book: Detailed Reference Manual for the Educational Computer Instruction Set for Laboratory Work

Table of Contents

- [Introduction](#)
 - [1. PDP-11 Model](#)
 - [1.1 General Purpose Registers](#)
 - [1.2 Processor Status Word \(PSW\)](#)
 - [1.3 Memory Access and Bus Address Distribution](#)
 - [1.4 Data Exchange Between External Devices and the Computer](#)
 - [2. Educational Computer Instruction Set and Addressing Modes](#)
 - [2.1 General Concepts](#)
 - [2.2 Single-Operand Instruction Format](#)
 - [2.3 Double-Operand Instruction Format](#)
 - [2.4 Direct Addressing Modes](#)
 - [2.5 Indirect \(Deferred\) Addressing Modes](#)
 - [2.6 Using the Program Counter \(PC\) as a General Purpose Register](#)
 - [3. Symbols Used in Instruction Descriptions](#)
 - [4. Execution of Byte Instructions](#)
 - [5. Single-Operand Instructions](#)
 - [6. Double-Operand Instructions](#)
 - [7. Program Control Instructions](#)
 - [7.1 Branch Instructions](#)
 - [7.2 Subroutine Call and Return Instructions](#)
 - [7.3 Interrupt Instructions](#)
 - [7.4 Processor Control Instructions](#)
-

Introduction

The purpose of practical classes in the "Computer Architecture" course is to study the fundamentals of computer organization and architecture using the software implementation of an educational c

omputer based on the PDP-11 processor family, which has become a classic for many educational publications.

During practical classes, students use the PDP-11 software model in the Windows operating system environment to complete most assignments.

1. PDP-11 Model

The PDP-11 model, written in C (MS Visual C++), runs in the Windows OS environment and has the following technical specifications:

- Number system for numbers and instructions - binary.
- Bit depth for numbers and instructions - 16 binary bits.
- Addressable RAM capacity - 32K 16-bit words.
- Number of general-purpose registers - 8.
- Instruction set: zero-operand, single-operand, double-operand.
- Addressing modes: register, register deferred, autoincrement, autoincrement deferred, autodecrement, autodecrement deferred, index, and index deferred.
- Processing of external and internal interrupts is performed using a Last-In-First-Out (LIFO) memory structure (stack).

The structural diagram of the educational computer is shown in Fig. 1.1.

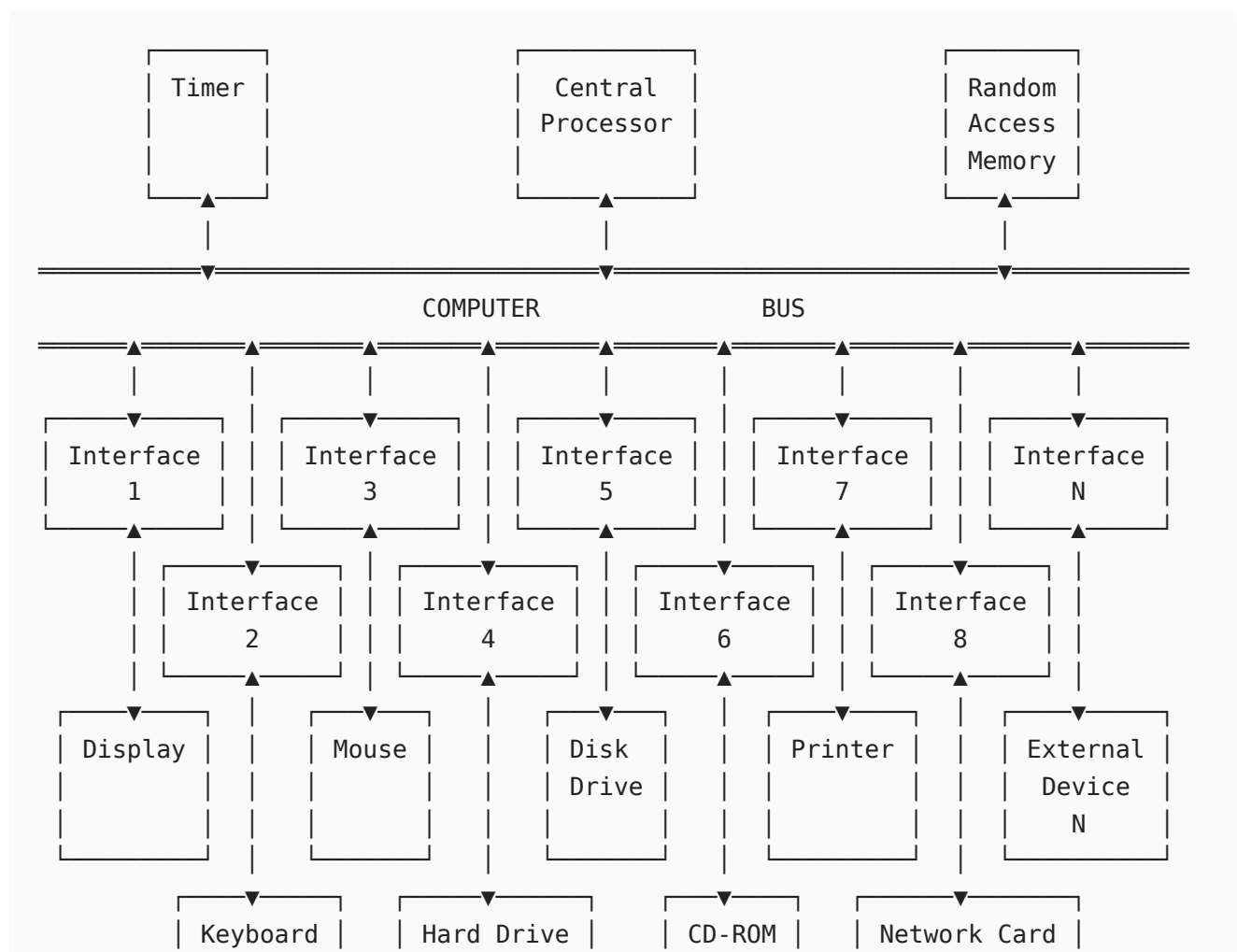


Fig. 1.1 Structural diagram of the educational computer

1.1 General Purpose Registers

The central processor module of the educational computer contains 16-bit general-purpose registers (GPRs) used for fetching operands and storing results during arithmetic and logical operations, similarly to memory cells and external device registers.

Two of the eight available general-purpose registers, R0 - R7, have a special purpose. Register R6 - the *Stack Pointer (SP)*, contains the address of the last filled cell in the stack.

Register R7 serves as the *Program Counter (PC)* and contains the address of the memory cell from which the processor fetches the next instruction to execute. Therefore, this register (R7) is typically used only for addressing and is not used as an accumulator for storing operands. The processor operates in such a way that after fetching an instruction from the computer memory at the address specified in R7 (PC), the contents of this register, i.e., the address of the current instruction, are automatically incremented by two. Thanks to this, after the execution of the current instruction, register R7 will contain the address of the next sequential instruction.

1.2 Processor Status Word (PSW)

The processor status register contains information about the current state of the processor. This is information (the Processor Status Word - PSW) about the values of the condition code bits used for program branching, which depend on the result of the instruction execution, the current processor priority, etc.

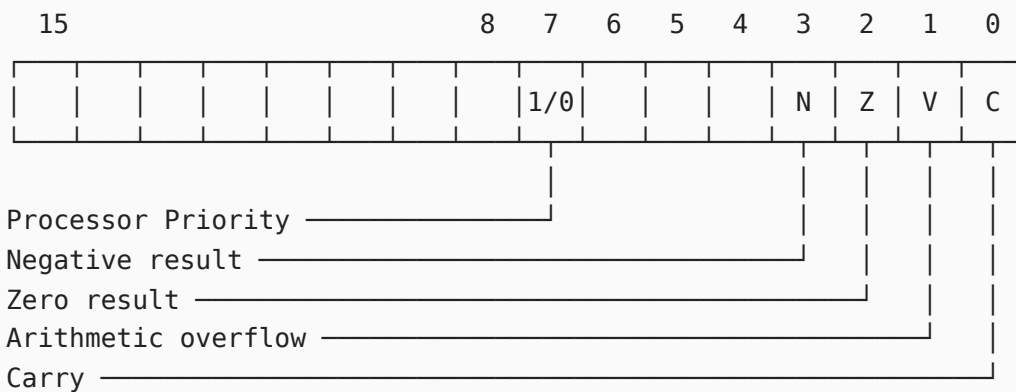
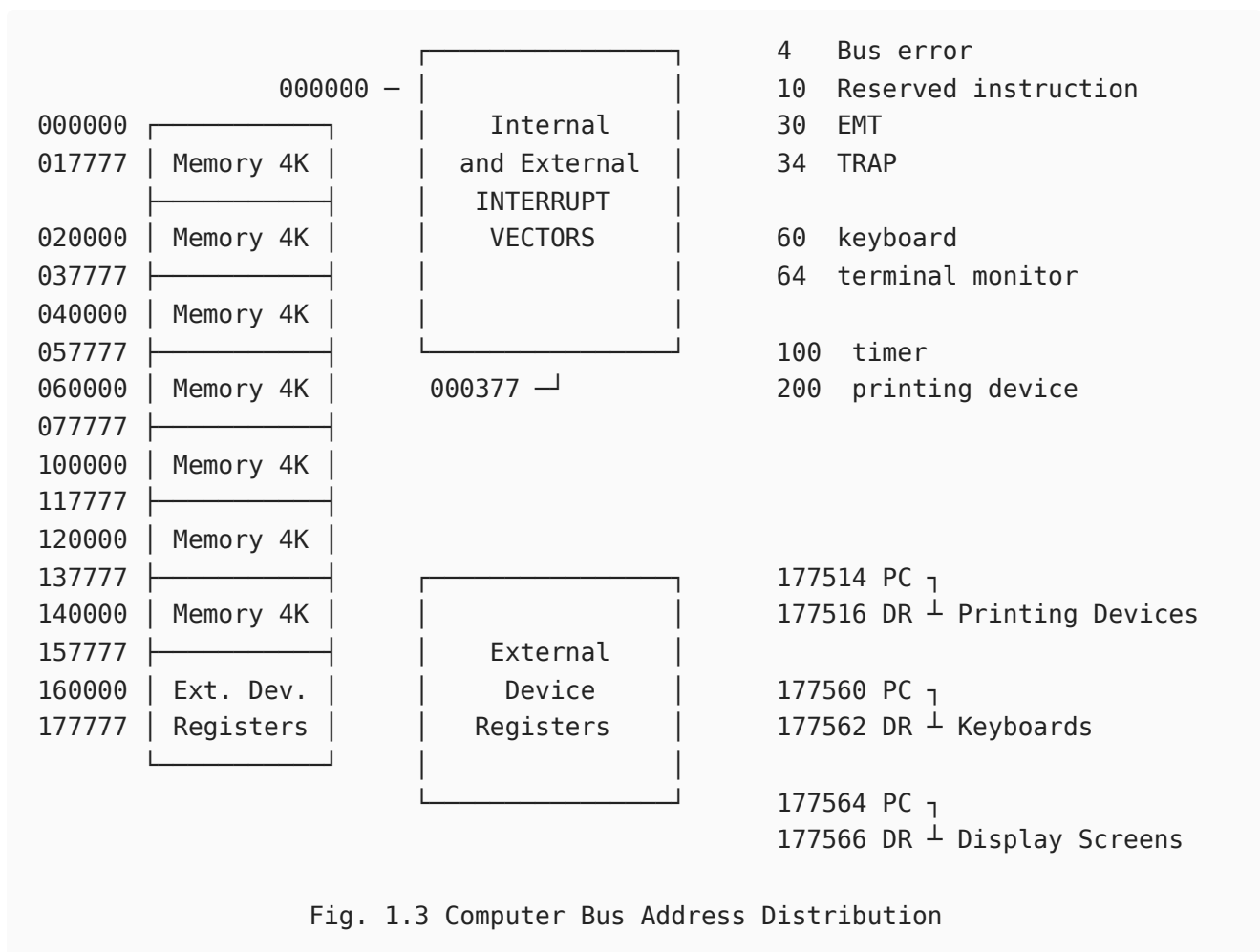


Fig. 1.2 Processor Status Word Format

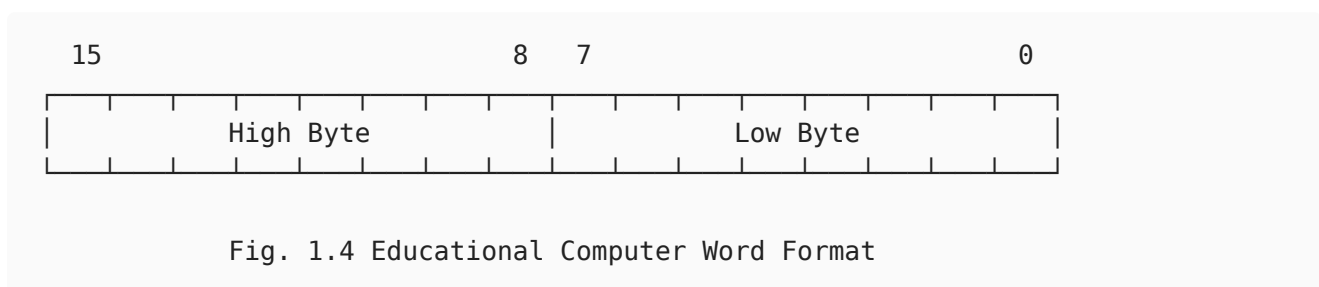
1.3 Memory Access and Bus Address Distribution

All data exchange and control signals between various devices and the computer are carried out through a single information transfer bus. A 16-bit address code allows access to 32K 16-bit cells. The top 4K addresses (28K-32K) are reserved for external device registers. The bus address distribution is shown in Fig. 1.3. All addresses are given in octal code. The letter "K" is used to denote a number equal to $2^{10} = 1024$.



The computer bus allows addressing up to 32K 16-bit words or 64K bytes. Memory cells from 000000 to 000376 are reserved for interrupt vectors, and using them for other purposes is not recommended. Each vector requires two 16-bit cells, so interrupt vector addresses are even and end in 0 or 4. The last 4K 16-bit addresses are usually allocated for external device registers, so the maximum physical memory capacity is 28K 16-bit words. However, the user is not obliged to use all the addresses in this space for this purpose and may be guided by necessity.

As shown in Fig. 1.4, a 16-bit machine word is divided into high and low bytes. Cells containing full words always have even addresses. The low bytes of words are stored in cells with even addresses, and the high bytes with odd addresses (Fig. 1.4).



WORD		addresses	BYTE	addresses
byte	byte			
High	Low	000000	Low	000000
High	Low	000002	High	000001
High	Low	000004	Low	000002

.	.	.	.	.
.	.	.	.	.
.	.	.	.	.
High	Low	017770	Low	017774
High	Low	017772	High	017775
High	Low	017774	Low	017776
High	Low	017776	High	017777

Fig. 1.5 Organization of computer memory by words and bytes for the first 4K addresses

1.4 Data Exchange Between External Devices and the Computer

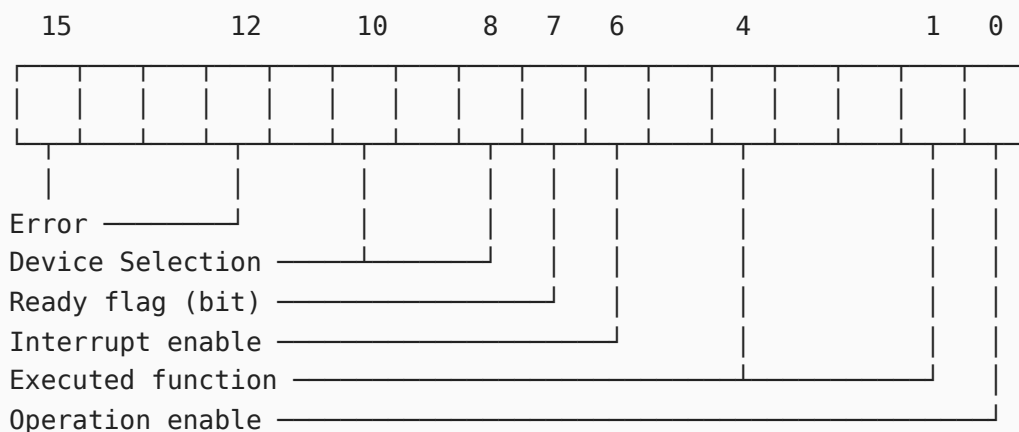
The computer bus provides three types of data exchange: programmed exchange, direct memory access (DMA), and interrupt-driven exchange. Information exchange between the central processor and external devices is performed using standard bus access cycles. To organize the exchange, each external device must have one or more registers (data registers, status registers, etc.), the addresses of which are determined by the user.

Typically, external device registers have even addresses, but byte instructions can be used to access any byte of a 16-bit register. Each external device can have several different registers.

The *Status Register (SR)* contains information about the operation performed by the external device, characterizes the state of the external device, and participates in interrupt provisioning operations.

The *Data Register (DR)* is used when exchanging data between the central processor and the external device.

Different bits in external device registers can perform different functions. Some of them can be used for both writing and reading information, while others are strictly read-only or write-only. A typical example of a bit used for both reading and writing is the interrupt enable bit in an external device's status register. An example of a write-only bit is the start bit, and a read-only bit is the error bit in an external device's status register. External device data registers are usually ordinary accumulator registers, and their format is determined solely by the user's requirements. The format of external device status registers is shown in Fig. 1.6. This format is not mandatory but is recommended to ensure the unification of operations performed when accessing external devices. Note that the status registers of most external devices have fewer than 16 bits.



2. Educational Computer Instruction Set and Addressing Modes

2.1 General Concepts

Computer instructions intended for data processing, in addition to the operation code, must somehow indicate the location (address) of this data (operands) in the computer's memory. Therefore, the operand addressing modes implemented in a specific computer, i.e., the ways of specifying the location of operands in memory within a machine instruction, are of great importance.

Addressing methods can be classified into direct and indirect. In direct addressing, the effective address is taken directly from the instruction or is calculated using a value specified in the instruction and the contents of a register.

Indirect addressing implies that the instruction contains an indirect address value, i.e., the address of the memory cell that holds the final effective address.

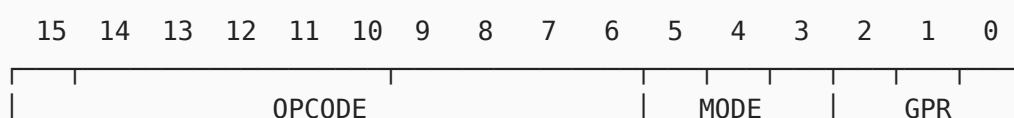
When implementing addressing methods, the central processor registers (GPRs) are heavily used. Below, we will use the term *address register* to denote any central processor register containing an address.

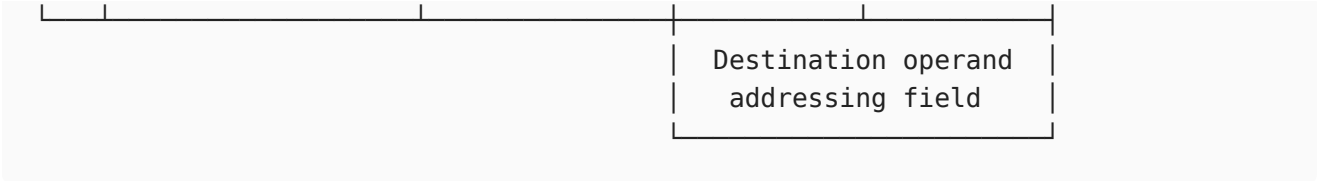
Data processing instructions can specify the location of one or more operands used when performing a specific operation. The educational computer uses single-operand and double-operand instructions. In this context, it is common to distinguish between a source operand and a destination operand. The source operand is the contents of a memory cell or register used during the execution of the specified operation, which remains unchanged during execution. The destination operand is a memory cell or GPR whose contents may also be used during execution and into which the result of the operation is placed (result receiver). In the examples below, the source operand is denoted by the letters *src* or *S* (source), and the destination operand is denoted by *dst* or *D* (destination). The instruction field containing the operation code will be denoted by the abbreviation *OPCODE*.

2.2 Single-Operand Instruction Format

Bits 15 - 06 contain the operation code, which determines the instruction to be executed. Bits 05 - 00 form a 6-bit field called the destination operand addressing field, which in turn consists of two subfields:

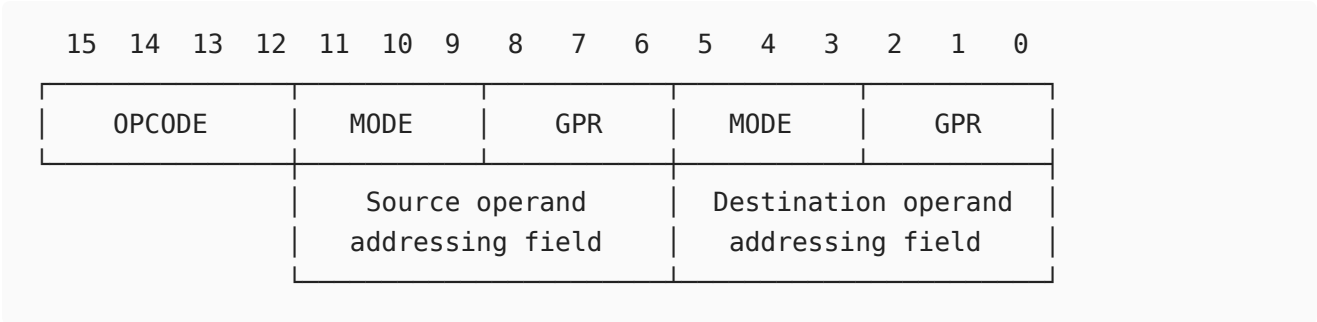
1. Bits 02 - 00 specify one of the eight GPRs used by the instruction;
2. Bits 05 - 03 specify the method of using the selected register (addressing mode). Bit 03 determines whether the addressing is direct or indirect.





2.3 Double-Operand Instruction Format

Operations on two operands (such as addition, moving, comparison) are performed using instructions that specify two addresses. Setting the bits in the source and destination operand addressing fields determines the addressing modes and general-purpose registers used. The format of a double-operand instruction is as follows:

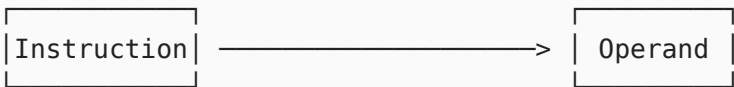


The source operand addressing field is used to fetch the source operand. The destination operand addressing field is used to fetch the destination operand and store the result. For example, in the `ADD A,B` instruction, the contents of cell "A" (source operand) are added to the contents of cell "B" (destination operand). After the addition operation is completed, cell "B" will contain the result of the operation, while the contents of cell "A" will remain unchanged.

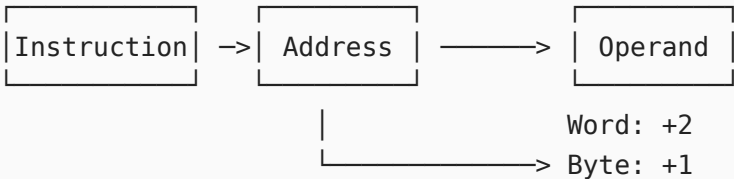
2.4 Direct Addressing Modes

Fig. 1.7 shows the sequences of operations when executing instructions with each of the four direct addressing modes. In register addressing, the operand is located directly in the selected register, which can be used as an accumulator. Since GPRs are implemented in the hardware of the central processor IC, they have a higher access speed than any other memory controlled by the processor. This advantage is especially evident in operations with variables that need to be accessed frequently.

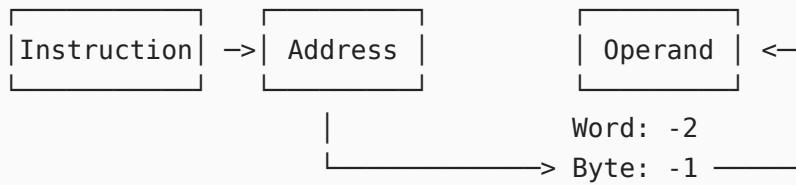
1. Register addressing mode (denoted as R, code $0_8 - 000_2$)



2. Autoincrement addressing mode (denoted as (R)+, code $2_8 - 010_2$)



3. Autodecrement addressing mode (denoted as $-(R)$, code $4_8 - 100_2$)



4. Index addressing mode (denoted as $X(R)$, code $6_8 - 110_2$)

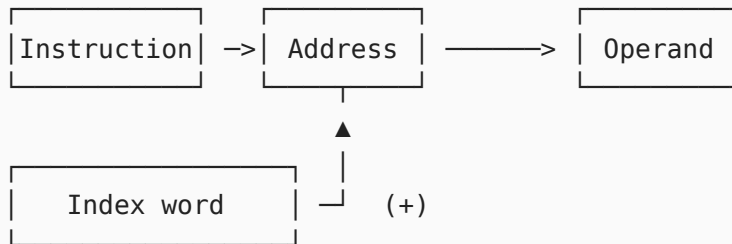


Fig. 1.7 Direct addressing modes

Register Addressing Mode

In register addressing, the operand is located directly in the register specified in the instruction.

Example 1.

Mnemonic	Octal Code	Description
INC R3	005203	Increment / Add one

Action: 1 is added to the contents of R3.

Autoincrement Addressing Mode

In autoincrement addressing, the contents of the selected register serve as the operand's address. After the operand is fetched, the contents of this register are automatically incremented to allow sequential access to the next memory cell. In byte operations, the increment is by 1; in full-word operations, it is by 2. The contents of R6 and R7 are always incremented by 2.

Autoincrement addressing is particularly useful for operations with arrays and stacks. With this method, you can access a table element and then increment the pointer to address the next element in the table.

Example 2.

Mnemonic	Octal Code	Description
CLR (R5)+	005025	Clear

Action: the cell whose address is in R5 is cleared, after which the address (contents of R5) is incremented by 2.

Before operation	After operation
20000/ 005025	20000/ 005025
30000/ 111116	30000/ 000000
R5/ 030000	R5/ 030002

Autodecrement Addressing Mode

Autodecrement addressing is also used for processing tabulated data. However, unlike autoincrement addressing, array cells are accessed in the reverse order. In this mode, the contents of the selected GPR are first decremented (by 1 for byte commands, by 2 for word commands), and then used as the effective address.

A combination of autoincrement and autodecrement addressing modes can be used efficiently when working with a stack.

Example 3.

Mnemonic	Octal Code	Description
INC - (R0)	005240	Increment / Add one

Action: the contents of R0 are decremented by 2 and used as the effective address. 1 is added to the operand fetched from the cell at this address.

Before operation	After operation
100/ 005240	100/ 005240
17774/ 000000	17774/ 000001
R0/ 017776	R0/ 017774

Index Addressing Mode

In index addressing, the effective address is determined as the sum of the contents of the selected GPR and the index word. This method allows random access to elements of data structures. The index word is stored in the memory cell immediately following the instruction word. In index addressing, the contents of the selected register can be used as a base for calculating a series of addresses.

Example 4.

Mnemonic	Octal Code	Description
CLR 200(R4)	005064	Clear

Action: the operand address is calculated by adding the code 200 to the contents of R4, after which the cell at the calculated address is cleared.

Before operation	After operation
1020/ 005064	1020/ 005064
1022/ 000200	1022/ 000200
1200/ 177777	1200/ 000000
R4/ 001000	R4/ 001000

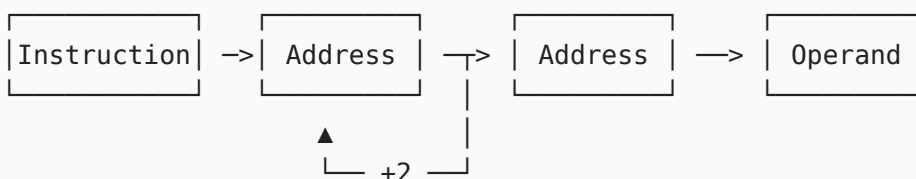
2.5 Indirect (Deferred) Addressing Modes

The four basic modes can be used in combination with indirect (deferred) addressing. While in the register mode the contents of the selected register are the operand, in the register deferred mode these contents serve as the address of the operand. In the other three deferred modes, the calculated address points only to the address of the operand, not to the operand itself. These methods are used when accessing tables containing addresses rather than operands (Fig. 1.8).

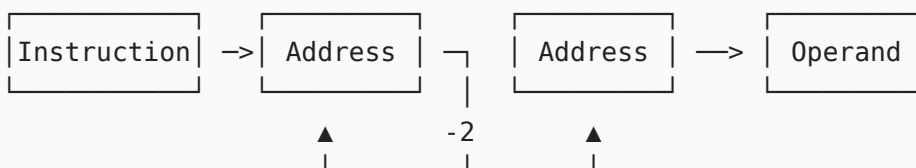
1. Register deferred addressing mode (denoted as @R, code $1_8 - 001_2$)



2. Autoincrement deferred addressing mode (denoted as @(R)+, code $3_8 - 011_2$)



3. Autodecrement deferred addressing mode (denoted as @-(R), code $5_8 - 101_2$)



4. Index deferred addressing mode (denoted as @X(R), code $7_8 - 111_2$)

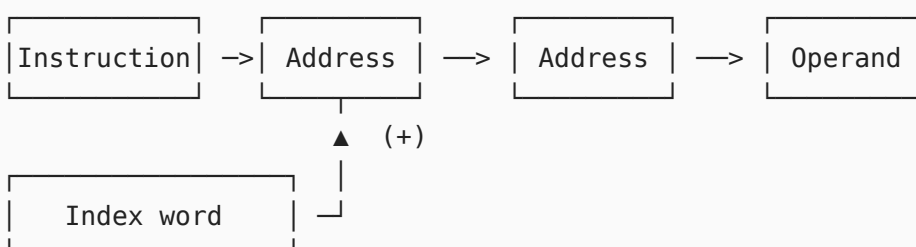


Fig. 1.8 Indirect (Deferred) Addressing Modes

2.6 Using the Program Counter (PC) as a General Purpose Register

The Program Counter R7 can be used with all addressing modes available in the microcomputer. However, it is most effectively used with only four of them. These addressing modes have received special names: immediate, absolute, relative, and relative deferred. The use of these modes enables the creation of position-independent programs, meaning their functionality is preserved when moved to any area in memory. The table below lists addressing modes using R7. It is important to understand that these four modes are analogous to those described above, but with R7 acting as the GPR. Addressing modes using the Program Counter greatly simplify the processing of unstructured data.

Octal Code	Binary Code	Mode Name	Function
2	010	Immediate	The operand is fetched from the cell following the instruction word.
3	011	Absolute	The address of the operand is fetched from the cell following the instruction word.
6	110	Relative	The operand is fetched from a cell whose address is determined as the sum of the contents of R7 and the cell following the instruction word.
7	111	Relative Deferred	The address of the operand is fetched from a cell whose address is determined as the sum of the contents of R7 and the cell following the instruction word.

Immediate Addressing Mode

The immediate addressing mode uses the symbol `#N`. It is equivalent to autoincrement addressing via the program counter R7. This method saves programmers time by allowing them to place a constant in the memory cell immediately following the instruction word.

Example 6.

Mnemonic	Octal Code	Description
ADD #10, R0	062700	Add

Action: the value 10 is added to the contents of R0. The result is stored in R0.

Before operation	After operation
1020/ 062700	1020/ 062700
1022/ 000010	1022/ 000010
R0/ 000020	R0/ 000030

Note. After fetching the instruction, the contents of R7 (the address of this instruction) are incremented by 2. Because the code 27 is written in the source operand address field, R7 is used as an address pointer when fetching the operand, after which its contents are incremented by 2 again to point to the next instruction.

Absolute Addressing Mode

The absolute addressing mode uses the symbol `@#A`. It is equivalent to autoincrement deferred addressing via R7. This method is convenient because the operand's address is an absolute address (i.e., it remains constant regardless of the program's location in memory).

Example 7.

Mnemonic	Octal Code	Description
CLR <code>@#1100</code>	005037	Clear

Action: the contents of the cell following the instruction are used as the operand's address (in this case, the effective address is the code 1100). The cell at address 1100 is cleared.

Before operation	After operation
20/ 005037	20/ 005037
22/ 001100	22/ 001100
1100/ 177777	1100/ 000000

Relative Addressing Mode

The relative addressing mode uses the symbol `X(PC)` or `A`, where X is the effective address relative to the program counter. This method is equivalent to index addressing via R7. The index word is stored in the cell following the instruction word and, when added to the contents of R7, yields the operand's address. This method is useful when writing programs that can be located in different memory locations, as the operand address is fixed relative to the contents of R7. When a program is relocated in memory, the operand moves by the same number of cells as the instruction itself.

Example 8.

Mnemonic	Octal Code	Description
INC A	005267	Increment / Add one

Action: "1" is added to the operand whose address is determined by adding the contents of R7 and the index word (000054).

Before operation	After operation
1020/ 005267	1020/ 005267
1022/ 000054	1022/ 000054
1024/	1024/
1100/ 000000	1100/ 000001

Relative Deferred Addressing Mode

The relative deferred addressing mode uses the symbol `@X(PC)` or `@A`, where X is the address of the cell containing the effective address relative to the program counter. This method is equivalent to

o index deferred addressing via PC.

3. Symbols Used in Instruction Descriptions

Each instruction description includes: a mnemonic, an octal code, instruction format, a binary code, an execution description, condition codes, special notes, and examples.

Symbol	Meaning	Symbol	Meaning
R	general purpose register	B	byte command
SP	stack pointer (R6)	PC	program counter (R7)
RS / PSR	processor status register	PSW	processor status word
SRC	source	SS	source operand addressing field
(SRC)	source operand	DD	destination operand addressing field
DST	destination	NN	offset (6 binary bits)
(DST)	destination operand	XXX	offset (8 binary bits)
()	cell contents	*	"exclusive OR"
&	logical multiplication ("AND")	V	logical addition ("OR")
/=/	not equal	=	equal
<>	not equal	$\bar{A}$	NOT A (negation)
$\hat{A}$	becomes equal	PUSH	push to stack
POP	pull from stack	*	multiplication
**	exponentiation		

4. Execution of Byte Instructions

Most computer instructions operate on both full words and bytes. Byte instructions using autoincrement or autodecrement addressing modes modify the contents of the specified register by "1" to access the next byte. Byte instructions in register addressing mode process the low byte of the selected register.

If the most significant bit of the instruction word (bit 15) is set to "1", it indicates a byte instruction. If a "0" is written in bit 15 of the instruction word, the instruction operates on a full word.

5. Single-Operand Instructions

<i>Clear</i>	CLR	0050DD
--------------	-----	--------

	CLRB	1050DD
<i>Action:</i>	$(DST) \leftarrow 0$	
<i>Description:</i>	A zero is written into the specified cell. For a byte instruction, a zero is written to the specified byte.	
<i>Condition Codes / Flags:</i>	N V Z C 0 1 0 0	

<i>Invert / Complement</i>	COM	0051DD
	COMB	1051DD
<i>Action:</i>	$(DST) \leftarrow \sim(DST)$	
<i>Description:</i>	The contents of the specified cell are replaced with its one's complement (each bit containing a 0 is set, and each bit containing a 1 is cleared). For a byte instruction, the operation is performed on the specified byte.	
<i>Condition Codes / Flags:</i>	N V Z C * 0 * 1 - N = 1, if <i>result</i> < 0 - Z = 1, if <i>result</i> = 0	

<i>Increment</i>	INC	0052DD
	INCB	1052DD
<i>Action:</i>	$(DST) \leftarrow (DST) + 1$	
<i>Description:</i>	One is added to the contents of the specified cell (or byte, if the instruction is a byte instruction).	
<i>Condition Codes / Flags:</i>	N V Z C * * * N = 1, if <i>result</i> < 0	

	• $Z = 1$, if <i>result</i> = 0 • $V = 1$, if <i>operand</i> = 077777 • C - not affected.
--	---

Decrement	DEC	0053DD
	DECB	1053DD
Action:	$(DST) \leftarrow (DST) - 1$	
Description:	One is subtracted from the contents of the specified cell (or the specified byte for byte instructions).	
Condition Codes / Flags:	N V Z C * * * • $N = 1$, if <i>result</i> < 0 • $Z = 1$, if <i>result</i> = 0 • $V = 1$, if <i>operand</i> = 100000 • C - not affected.	

Negate	NEG	0054DD
	NEGB	1054DD
Action:	$(DST) \leftarrow -(DST)$	
Description:	The contents of the specified cell (or byte for byte instructions) are replaced by their two's complement. Note that the number 100000 is replaced by itself, since there is no corresponding positive number.	

<i>Condition Codes / Flags:</i>	N V Z C * * * * • N = 1, if <i>result</i> < 0 • Z = 1, if <i>result</i> = 0 • V = 1, if <i>operand</i> = 100000 • C = 0, if <i>result</i> = 0
---------------------------------	--

<i>Test</i>	TST	0057DD
	TSTB	1057DD
<i>Action:</i>	(DST) ← (DST)	
<i>Description:</i>	Depending on the contents of the specified cell (or byte for byte instructions), the N and Z flags are set or cleared.	
<i>Condition Codes / Flags:</i>	N V Z C * 0 * 0 • N = 1, if <i>result</i> < 0 • Z = 1, if <i>result</i> = 0	

Scaling numbers by powers of 2 is performed using arithmetic shift instructions: ASR for arithmetic shift right and ASL for arithmetic shift left. The sign bit of the operand (bit 15) is replicated during an arithmetic shift right. A zero is loaded into the least significant bit during an arithmetic shift left. Information shifted beyond the C bit is lost.

<i>Arithmetic shift right</i>	ASR	0062DD
	ASRB	1062DD

Action:	(DST) ← (DST) shifted one position to the right
Description:	All bits of the operand are shifted right by one position. The contents of the sign bit are replicated. The C bit is loaded with the contents of the operand's least significant bit. Thus, ASR or ASRB performs division of a signed number by 2.
Condition Codes / Flags:	N V Z C * * * * • N = 1, if <i>result</i> < 0 • Z = 1, if <i>result</i> = 0 • V = N*C (after shift) • C = contents of the least significant bit of the specified cell

Arithmetic shift left	ASL	0063DD
	ASLB	1063DD
Action:	(DST) ← (DST) shifted one position to the left	
Description:	All bits of the operand are shifted left by one position. A zero is written to the least significant bit of the result. The C bit is loaded with the contents of the operand's most significant bit. Thus, ASL or ASLB performs multiplication of a signed number by 2.	
Condition Codes / Flags:	N V Z C * * * * • N = 1, if <i>result</i> < 0 • Z = 1, if <i>result</i> = 0 • V = N*C (after shift) • C = contents of the most significant bit of the specified cell	

Rotate instructions are used to facilitate sequential checking and bitwise processing of an operand. They operate on the operand word and the C bit as if they were a 17-bit circular shift register.

<i>Rotate right</i>	ROR	0060DD
	RORB	1060DD
Action:	(DST) ← (DST) rotated one position to the right	
Description:	All bits of the operand are cyclically shifted one position to the right. The contents of the least significant bit are loaded into the C bit, and the previous contents of the C bit are loaded into the most significant bit of the result.	
Condition Codes / Flags:	N V Z C * * * * • N = 1, if <i>result</i> < 0 • Z = 1, if <i>result</i> = 0 • V = N*C (after shift) • C = contents of the least significant bit of the operand	

<i>Rotate left</i>	ROL	0061DD
	ROLB	1061DD
Action:	(DST) ← (DST) rotated one position to the left	
Description:	All bits of the operand are cyclically shifted one position to the left. The contents of the most significant bit are loaded into the C bit, and the previous contents of the C bit are loaded into the least significant bit of the result.	
Condition Codes / Flags:		

	N V Z C * * * * • $N = 1$, if <i>result</i> < 0 • $Z = 1$, if <i>result</i> = 0 • $V = N * C$ (after shift) • C = contents of the most significant bit of the operand
--	---

6. Double-Operand Instructions

Using double-operand instructions saves machine time and reduces the number of instructions in a program. The list of double-operand instructions contains four arithmetic and four logical instructions.

Move	MOV	01SSDD
	MOVB	11SSDD
Action:	(DST) ← (SRC)	
Description:	The source operand (SRC) is moved to the destination operand address. The previous contents of the DST cell are lost, and the contents of the SRC cell remain unchanged. During byte operations, the MOVB instruction using register addressing (the only one among byte instructions) extends the most significant bit of the low byte (sign extension). All bits of the high byte are set or cleared depending on whether the most significant (sign) bit of the low byte is set or cleared. In other cases, MOVB operates on bytes in the same way MOV operates on words.	
Condition Codes / Flags:	N V Z C * 0 * • $N = 1$, if (SRC) < 0 • $Z = 1$, if (SRC) = 0	

	• C - not affected
--	--

Compare	CMP	02SSDD
	CMPB	12SSDD
Action:	(SRC) - (DST)	
Description:	The source and destination operands are compared, and as a result of the comparison, the condition codes are updated, which can then be used for conditional branch instructions. Both operands remain unchanged. A comparison instruction is usually followed by a conditional branch instruction. Note that, unlike the subtract instruction, during the execution of the CMP instruction, the operands are swapped, i.e., it computes (SRC) - (DST), not (DST) - (SRC).	
Condition Codes / Flags:	N V Z C * * * * • N = 1, if <i>result</i> < 0 • Z = 1, if <i>result</i> = 0 • V = 1, if <i>arithmetic overflow</i> • C = 1, if <i>carry from the most significant bit</i>	

Add	ADD	06SSDD
Action:	(DST) ← (SRC) + (DST)	
Description:	The source operand (SRC) is added to the destination operand (DST), and the result is stored at the destination operand address. The original contents of (DST) are lost. The contents of (SRC) remain unchanged. Addition is performed using two's complement binary arithmetic. (There is no byte equivalent.)	
Condition Codes / Flags:		

	N V Z C * * * * • N = 1, if <i>result</i> < 0 • Z = 1, if <i>result</i> = 0 • V = 1, if <i>arithmetic overflow</i> • C = 1, if <i>carry</i>
--	--

Subtract	SUB	16SSDD
Action:	(DST) ← (DST) - (SRC)	
Description:	The source operand (SRC) is subtracted from the destination operand (DST), and the result is written to the DST address. The original contents of DST are lost, while the contents of SRC remain unchanged. (There is no byte equivalent.)	
Condition Codes / Flags:	N V Z C * * * * • N = 1, if <i>result</i> < 0 • Z = 1, if <i>result</i> = 0 • V = 1, if <i>arithmetic overflow</i> • C = 1, if <i>carry from the most significant bit</i>	

Logical Instructions

Of the four logical instructions, three have the same format as the double-operand arithmetic instructions. The fourth instruction has a specific format. Logical instructions allow bitwise data processing.

Bit test	BIT	03SSDD
	BITB	13SSDD
Action:	(DST) & (SRC)	
Description:	Performs a logical "AND" operation on (SRC) and (DST) and updates the condition codes accordingly. Neither operand changes its value.	
Condition Codes / Flags:	N V Z C * 0 * • N = 1, if <i>result</i> < 0 • Z = 1, if <i>result</i> = 0 • C - not affected	

Bit clear	BIC	04SSDD
	BICB	14SSDD
Action:	(DST) ← ~(SRC) & (DST)	
Description:	Clears each bit in the (DST) operand that corresponds to a set bit in the (SRC) operand. The original contents of DST are lost; the contents of SRC are not modified.	
Condition Codes / Flags:	N V Z C * 0 * • N = 1, if <i>result</i> < 0 • Z = 1, if <i>result</i> = 0 • C - not affected	

Bit set (Logical OR)	BIS	05SSDD
	BISB	15SSDD
Action:	$(DST) \leftarrow (SRC) \vee (DST)$	
Description:	A logical "OR" operation is performed on the contents of SRC and DST, and the result is stored at the DST address. Bits in DST are set to "1" if the corresponding bits in (SRC) are "1". The previous contents of DST are lost, and the contents of SRC remain unchanged.	
Condition Codes / Flags:	N V Z C * 0 * • N = 1, if <i>result</i> < 0 • Z = 1, if <i>result</i> = 0 • C - not affected	

Exclusive OR	XOR	74RDD
Action:	$(DST) \leftarrow R * (DST)$	
Description:	An exclusive OR operation is performed on the contents of the specified register R and the contents of DST. The result is stored in DST. The contents of register R are not modified.	
Condition Codes / Flags:	N V Z C * 0 * • N = 1, if <i>result</i> < 0 • Z = 1, if <i>result</i> = 0 • C - not affected	

7. Program Control Instructions

Program control instructions include branch instructions, subroutine calls, returns from subroutines, unconditional jumps, and others.

7.1 Branch Instructions

These instructions cause a branch to an address calculated as the sum of the offset (multiplied by 2) and the current contents of the program counter R7, if the branch condition is met.

The offset indicates how many memory cells away from the current program counter value to jump in either direction. Since words have even addresses, to get the true effective address, the offset must be multiplied by two before being added to the program counter R7, which always points to a word. The most significant bit of the offset (bit 7) is the sign bit. If it is set to 1, the offset is negative, and branching occurs toward lower addresses (backward). If bit 7 contains a 0, the offset is positive, and branching occurs toward higher addresses (forward). An 8-bit offset allows branching backward by a maximum of 200_8 words from the word currently pointed to by PC, and forward by 177_8 words.

New PC contents = current PC contents + 2 * XXX (offset), where current PC contents = branch instruction address + 2.

<i>Unconditional branch</i>	BR	000400 + XXX
<i>Action:</i>	$(PC) \leftarrow (PC) + 2 * XXX$	
<i>Description:</i>	Transfers program control via a single instruction to a cell whose address lies within a limited range.	
<i>Condition Codes / Flags:</i>	N V Z C Not affected	

Simple Conditional Branches

<i>Branch if not equal (to zero)</i>	BNE	001000 + XXX
<i>Action:</i>	$(PC) \leftarrow (PC) + 2 * XXX, \text{ if } Z = 0$	
<i>Description:</i>	Checks the state of the Z bit and branches if it is cleared. The BNE instruction is the opposite of the BEQ instruction. Together with the BIT instruction, it is used to verify that the set bits of the source operand match the set bits of the destination operand. Generally, it is used to check that the result of the previous operation is not equal to zero.	
<i>Condition Codes / Flags:</i>	N V Z C Not affected	

Branch if equal (to zero)	BEQ	001400 + XXX
Action:	$(PC) \leftarrow (PC) + 2 * XXX, \text{ if } Z = 1$	
Description:	Checks the state of the Z bit and branches if it is set. The BEQ instruction is the opposite of the BNE instruction. Together with the CMP instruction, BEQ is used to check the equality of two values. Together with the BIT instruction, it is used to verify that the cleared bits of the source operand match the set bits of the destination. Generally, this instruction is used to test if the result of the previous operation equals zero.	
Condition Codes / Flags:	N V Z C Not affected	

Branch if plus	BPL	100000 + XXX
Action:	$(PC) \leftarrow (PC) + 2 * XXX, \text{ if } N = 0$	
Description:	Checks the N bit and branches if it is cleared. Together with the TSTB instruction, it is used to check if bit 7 (the ready flag) of a peripheral device status register is set. Generally, this instruction is used to verify that the result of the previous operation is positive. The BPL instruction is the opposite of the BMI instruction.	
Condition Codes / Flags:	N V Z C Not affected	

Branch if minus	BMI	100400 + XXX
Action:	$(PC) \leftarrow (PC) + 2 * XXX, \text{ if } N = 1$	
Description:	Checks the state of the N bit and branches if it is set. Used to check the sign (most significant bit) of the result of the previous operation.	
Condition Codes / Flags:	N V Z C Not affected	

Branch if overflow clear	BVC	102000 + XXX
Action:	$(PC) \leftarrow (PC) + 2 * XXX, \text{ if } V = 0$	
Description:	Checks the state of the V bit and branches if it is cleared.	
Condition Codes / Flags:	N V Z C Not affected	

Branch if overflow set	BVS	102400 + XXX
-------------------------------	------------	---------------------

<i>Action:</i>	$(PC) \leftarrow (PC) + 2 * XXX, \text{ if } V = 1$
<i>Description:</i>	Checks the state of the V bit and branches if it is set. BVS is used to detect arithmetic overflow resulting from the execution of the previous operation. BVS is the opposite of the BVC instruction.
<i>Condition Codes / Flags:</i>	N V Z C Not affected

<i>Branch if carry clear</i>	BCC	103000 + XXX
<i>Action:</i>	$(PC) \leftarrow (PC) + 2 * XXX, \text{ if } C = 0$	
<i>Description:</i>	Checks the C bit and branches if it is cleared.	
<i>Condition Codes / Flags:</i>	N V Z C Not affected	

<i>Branch if carry set</i>	BCS	103400 + XXX
<i>Action:</i>	$(PC) \leftarrow (PC) + 2 * XXX, \text{ if } C = 1$	
<i>Description:</i>	Checks the C bit and branches if it is set. BCS is used to test for a carry condition resulting from the previous operation. The BCS instruction is the opposite of the BCC instruction.	
<i>Condition Codes / Flags:</i>	N V Z C Not affected	

Conditional Branches Based on Signed Operations

Special combinations of condition codes are tested using conditional branch instructions based on arithmetic operation results. These instructions are used to check the results of commands where operands are treated as signed binary numbers. Note that the difference between comparing signed and unsigned numbers stems from their different representation in two's complement arithmetic. For unsigned 16-bit numbers, the sequence is as follows:

```
highest 177777
077776
.....
000002
000001
lowest 000000
```

Conditional branch commands based on the result of an operation on numbers are: BGE, BLT, BGT, BLE.

Branch if greater than or equal (to zero)	BGE	002000 + XXX
Action:	$(PC) \leftarrow (PC) + 2 * XXX, \text{ if } N*V = 0$	
Description:	Causes a branch if both the N and V condition code bits are either set or cleared (i.e., if the result of an exclusive OR operation on the N and V bits is 0). Thus, the BGE instruction will always branch if it follows the addition of two positive numbers. The BGE instruction will also branch on a zero result.	
Condition Codes / Flags:	N V Z C Not affected	

Branch if less than (zero)	BLT	002400 + XXX
Action:	$(PC) \leftarrow (PC) + 2 * XXX, \text{ if } N*V = 1$	
Description:	Causes a branch if the result of an exclusive OR operation on the N and V bits is 1. The BLT instruction is the opposite of the BGE instruction. Thus, the BLT instruction will always branch if it follows the addition of two negative numbers, even if an overflow occurs. In particular, the BLT instruction will always branch if it follows a compare (CMP) command of a negative source operand and a positive destination operand, even if overflow occurred. The BLT instruction will never branch if it follows a compare (CMP) command of a positive source operand and a negative destination operand. It will also not branch if the result of the previous operation equals zero without overflow.	
Condition Codes / Flags:	N V Z C Not affected	

Branch if greater than (zero)	BGT	003000 + XXX
Action:	$(PC) \leftarrow (PC) + 2 * XXX, \text{ if } Z \vee (N*C) = 0$	
Description:	The BGT instruction is similar to the BGE instruction, except that BGT does not cause a branch on a zero result.	
Condition Codes / Flags:	N V Z C Not affected	

Branch if less than or equal (to zero)	BLE	003400 + XXX
Action:	$(PC) \leftarrow (PC) + 2 * XXX, \text{ if } Z \vee (N*C) = 1$	
Description:	The BLE instruction is similar to the BLT instruction, but it also causes a branch on a zero result.	

<i>Condition Codes / Flags:</i>	N V Z C Not affected
---------------------------------	-------------------------

Conditional Branches Based on Unsigned Operations

Unsigned conditional branch instructions provide methods for verifying the result of comparison operations on operands treated as unsigned values.

<i>Branch if higher</i>	BHI	101000 + XXX
<i>Action:</i>	$(PC) \leftarrow (PC) + 2 * XXX$, if $C = 0$ and $Z = 0$	
<i>Description:</i>	Causes a branch if the previous operation did not cause a carry or a zero result. This occurs during CMP comparison operations when the source operand is greater than the destination operand.	
<i>Condition Codes / Flags:</i>	N V Z C Not affected	

<i>Branch if higher or same</i>	BHIS	103000 + XXX
<i>Action:</i>	$(PC) \leftarrow (PC) + 2 * XXX$, if $C = 0$	
<i>Description:</i>	In its operation, the BHIS instruction is identical to the BCC instruction. A different mnemonic is introduced solely for a different conceptual use of the instruction.	
<i>Condition Codes / Flags:</i>	N V Z C Not affected	

<i>Branch if lower</i>	BLO	103400 + XXX
<i>Action:</i>	$(PC) \leftarrow (PC) + 2 * XXX$, if $C = 1$	
<i>Description:</i>	In its operation, the BLO instruction is identical to the BCS instruction. A different mnemonic is introduced solely for a different conceptual use of the instruction.	
<i>Condition Codes / Flags:</i>	N V Z C Not affected	

<i>Branch if lower or same</i>	BLOS	101100 + XXX
<i>Action:</i>	$(PC) \leftarrow (PC) + 2 * XXX$, if $C = 1$ or $Z = 1$	
<i>Description:</i>	In its operation, the BLOS instruction is the opposite of BHI. A branch will occur if the source operand is less than or equal to the destination operand (unsigned).	

Condition Codes / Flags:	N V Z C Not affected
--------------------------	-------------------------

<i>Jump</i>	JMP	0001DD
Action:	(PC) ← (DST)	
Description:	The JMP instruction enables a transfer of control to any program instruction (not limited to the bounds of +177 and -200 words like the BR instruction) using all addressing modes, except register addressing. The use of register addressing triggers a program interrupt on an <i>illegal instruction</i> condition via vector address 4. Indirect addressing mode can be applied and causes program control transfer to the address contained in the specified register. Note that instructions are full words and therefore must be fetched from cells with even addresses. The JMP instruction with an index deferred addressing mode allows transferring control to an address that is an element of an address table.	
Condition Codes / Flags:	N V Z C Not affected	

7.2 Subroutine Call and Return Instructions

These instructions allow for automatic nesting of subroutines, exiting from a subroutine, and repeated entry into a subroutine. Subroutines may contain calls to other subroutines (or to themselves) without requiring special programming to save return addresses. The procedure for calling and returning from a subroutine does not modify the subroutine itself. This allows the same subroutine to be used by multiple processes handling program interrupts.

<i>Jump to subroutine</i>	JSR	004RDD
Action:	(TMP) ← (DST) — write the destination contents into an internal processor register	
	PUSH (SP) ← (R) — push the contents of the specified register onto the stack	
	(R) ← (PC) — the program counter saves the address of the instruction following JSR into register R	
	(PC) ← (TMP) — load the starting address of the subroutine into the program counter	
Description:	When executing the JSR instruction, the old contents of the specified register (linkage pointer) are automatically pushed onto the stack, and the new linkage information is loaded into the register. Thus, calling a subroutine, nested within another subroutine to any depth, is carried out using the linkage pointer register. There is no need to specify a maximum call depth for a given subroutine or to include instructions for saving and restoring the linkage pointer in each subroutine. A call to a subroutine using	

	the JSR instruction can be executed with autoincrement addressing (if each subsequent entry to the subroutine is made through a cell whose address is 2 greater than the previous one) or index addressing (if entry to the subroutine is made via addresses arranged in arbitrary order). Both of these modes can also be deferred (indirect).
<i>Condition Codes / Flags:</i>	N V Z C Not affected

<i>Return from subroutine</i>	RTS	00020R
<i>Action:</i>	(PC) ← (R)	
	(R) ← (SP) POP	
<i>Description:</i>	The contents of register R are loaded into the program counter, after which the top element of the stack is popped and sent to the specified register. The return from a subroutine is usually executed via the exact same register used to call it. Thus, entering and exiting a subroutine called with the JSR PC, DST instruction is performed by the RTS PC instruction, while exiting a subroutine called by the JSR R5, DST instruction using any addressing mode is performed by the RTS R5 instruction.	
<i>Condition Codes / Flags:</i>	N V Z C Not affected	

<i>Subtract one and branch</i>	SOB	077RNN
<i>Action:</i>	(R) ← (R) - 1	
	(PC) ← (PC) - 2 * NN , if result <> 0	
	(PC) ← (PC) , if result = 0	
<i>Description:</i>	The contents of the register are decremented by one. If the result is not zero, the program counter is loaded with a new value determined by subtracting twice the offset from the current contents of the program counter. In the SOB instruction, the offset is a 6-bit positive number. This instruction can be effectively used to organize various types of counters and loops. It should be noted that the SOB instruction cannot be used to transfer control forward.	
<i>Condition Codes / Flags:</i>	N V Z C Not affected	

7.3 Interrupt Instructions

Interrupt instructions provide the ability to call input-output control programs, debugging programs, and user-developed programs. When an interrupt occurs, the current contents of the program counter and the contents of the processor status register are pushed onto the stack. The new contents

of the program counter and the processor status register are loaded from an interrupt vector consisting of two words. When exiting an interrupt, the RTI and RTT instructions are used, which restore the PC and PSR by popping their previous contents from the stack. Interrupt vectors are located at fixed addresses assigned to each type of interrupt.

Emulator trap	EMT	104000 - 104377
<i>Action:</i>	PUSH (SP) ← (RS)	
	PUSH (SP) ← (PC)	
	(PC) ← (30)	
	(RS) ← (32)	
<i>Description:</i>	EMT instructions have operation codes from 104000 to 104377, which can be used to pass information to a simulation program (i.e., information about a function to be executed). The interrupt vector for the EMT instruction is located at address 30. The new PC contents are taken from the cell at address 30, and the new PSR contents - from the cell at address 32.	
<i>Condition Codes / Flags:</i>	N V Z C Not affected	

Return from interrupt	RTI	000002
<i>Action:</i>	(PC) ← (SP) POP	
	(RS) ← (SP) POP	
<i>Description:</i>	The RTI instruction is used to exit routines that service external and internal interrupts. The contents of the program counter and processor status register are restored using the stack.	
<i>Condition Codes / Flags:</i>	N V Z C Not affected	

7.4 Processor Control Instructions

Halt	HALT	000000
<i>Action:</i>	(PC) ← (SP) POP	
	(RS) ← (SP) POP	
<i>Description:</i>	The central processor enters console terminal polling mode. The <i>program counter</i> saves the address of the instruction that is to be executed <i>next</i> .	
<i>Condition Codes / Flags:</i>	N V Z C Not affected	